

AI FAQ Assistant API

3. Project Design Phase

1. Introduction

This document describes the system design for the AI FAQ Assistant API, translating the requirements defined in the Requirement Analysis phase into a concrete architecture, module breakdown, database schema, and API contract. The design aims to be modular, so that individual components — particularly the NLP matching engine — can be upgraded independently over time.

2. Design Goals

- Keep the API layer independent of the specific NLP technique used, so the matching engine can be swapped or upgraded later
- Store all FAQ content, logs, and feedback in a structured, queryable database for analytics
- Separate admin (content management) concerns from end-user (question-asking) concerns
- Design for reasonable scale from the start, avoiding designs that only work for a handful of FAQs

3. System Architecture Overview

The system follows a layered architecture consisting of five logical layers, described below.

Layer	Responsibility
Client Layer	Any consumer of the API — website, mobile app, chatbot widget, or internal tool
API Layer	Receives HTTP requests, validates input, applies authentication, and routes to the correct handler
NLP Matching Engine	Converts text into vector embeddings and computes similarity to find the closest FAQ matches

Data Layer	Relational database storing FAQs, categories, query logs, feedback, and user accounts
Admin Layer	Management functions (secured) for maintaining FAQ content and reviewing analytics

At a high level, a client sends a question to the API layer, which forwards it to the NLP Matching Engine. The engine compares the question against stored FAQ embeddings in the Data Layer and returns the best candidate answer(s). The API layer then formats and returns the response to the client, while simultaneously logging the interaction for analytics.

4. Module Design

Module	Responsibility
Query Handler	Receives incoming questions, validates input, orchestrates the matching process
NLP Matching Engine	Converts text to embeddings/vectors and finds the closest FAQ matches using similarity scoring
FAQ Management	CRUD operations for FAQs and categories, restricted to admin users
Auth Module	Handles admin login, token issuance, and route-level access control
Analytics / Logging	Records queries, matched results, and confidence scores for reporting
Feedback Module	Captures and stores helpful / not-helpful ratings tied to specific queries

5. Database Design

The relational schema is designed around five core tables. Primary and foreign key relationships ensure that logs and feedback remain traceable back to the specific FAQ and query that produced them.

Table	Key Columns	Notes
users	id (PK), name, email, password_hash, role	role distinguishes admin vs. regular accounts
categories	id (PK), name, description	Used to group related FAQs
faqs	id (PK), category_id (FK), question, answer, keywords, embedding_vector, created_at	embedding_vector stores the precomputed NLP representation
query_logs	id (PK), user_query, matched_faq_id (FK), confidence_score, created_at	One row per question asked through the API
feedback	id (PK), query_log_id (FK), is_helpful, comment, created_at	Linked to the specific query it rates

5.1 Entity Relationship Summary

A category can have many FAQs (one-to-many). Each FAQ can appear in many query logs as the matched result (one-to-many). Each query log can have at most one feedback entry (one-to-one/optional). This structure keeps the schema simple while supporting all required analytics.

6. API Endpoint Design

The API is designed as a REST interface using standard HTTP methods and JSON payloads. The table below lists the primary endpoints; a full contract will be documented using Swagger/OpenAPI during development.

Method	Endpoint	Access	Description
POST	/api/ask	Public	Submit a question, receive the best-matching answer and confidence score
GET	/api/faqs	Admin	List all FAQs, optionally filtered by category
POST	/api/faqs	Admin	Create a new FAQ entry
PUT	/api/faqs/{id}	Admin	Update an existing FAQ entry
DELETE	/api/faqs/{id}	Admin	Delete an FAQ entry

POST	/api/feedback	Public	Submit feedback on a previously returned answer
POST	/api/auth/login	Public	Admin login; returns an access token
GET	/api/analytics/top-queries	Admin	Retrieve most frequently asked and unanswered questions

7. Data Flow Description

The following steps describe the end-to-end flow for a typical question-answer interaction:

- 1. The client sends a question to POST /api/ask.
- 2. The Query Handler validates the request and forwards the text to the NLP Matching Engine.
- 3. The engine converts the text into an embedding and compares it against stored FAQ embeddings.
- 4. Candidate matches are ranked by similarity score.
- 5. If the top score exceeds the configured confidence threshold, that FAQ's answer is returned; otherwise a fallback message is returned.
- 6. The Analytics/Logging module records the query, match result, and score.
- 7. The client may optionally call POST /api/feedback to rate the answer.

8. User Interface / Interaction Design Notes

Although this project is primarily an API (not a full front-end application), a minimal reference interaction is designed for demonstration and admin purposes:

- A simple admin panel or Postman collection to exercise FAQ management endpoints
- A basic chat-style demo page showing a question box and the returned answer with a thumbs-up/down feedback control

9. Design Considerations and Trade-offs

- Confidence threshold is made configurable so it can be tuned without code changes as more data becomes available
- FAQ embeddings are pre-computed and cached at creation/update time rather than recalculated on every request, improving response time
- The NLP engine is accessed through an internal interface so it can be replaced with a different model or provider with minimal impact on the rest of the system
- Soft-delete is considered for FAQs (marking as inactive rather than removing) to preserve historical analytics accuracy

10. Security Design Notes

- Passwords are stored as salted hashes, never in plain text
- Admin endpoints require a valid authentication token (e.g., JWT) checked by middleware before the request reaches the FAQ Management module
- All user input is validated and sanitized before being used in database queries to prevent injection attacks

11. Sequence Description: Ask-a-Question Flow

The following sequence describes, step by step, the interaction between components for a single question-answer cycle, as would typically be shown in a sequence diagram:

- Client -> API Layer: sends POST /api/ask with the question text
- API Layer -> Query Handler: forwards the validated request
- Query Handler -> NLP Matching Engine: requests the best-matching FAQ(s)
- NLP Matching Engine -> Data Layer: retrieves stored FAQ embeddings for comparison
- NLP Matching Engine -> Query Handler: returns ranked candidate matches with scores
- Query Handler -> Analytics/Logging: records the query and result
- API Layer -> Client: returns the final answer and confidence score

12. Class-Level Design Notes

At the code level, the design anticipates the following key classes/modules, which map directly onto the modules described in Section 4:

Class / Module	Key Responsibilities
FaqController	Handles HTTP requests for FAQ CRUD operations
FaqService	Contains business logic for validating and persisting FAQ data
MatchingService	Wraps the NLP engine; exposes a simple 'findBestMatch(question)' function
AuthController	Handles login requests and token issuance
AuthMiddleware	Verifies tokens on protected routes before passing control onward
QueryLogRepository	Handles reading and writing query log records

13. Error Handling Design

Scenario	Response Design
Missing or empty question field	HTTP 400 with a clear validation message
No confident FAQ match found	HTTP 200 with a fallback message and null/low confidence score
Invalid or missing admin token	HTTP 401 Unauthorized
Admin token valid but insufficient role	HTTP 403 Forbidden
Unexpected server error	HTTP 500 with a generic message; details logged internally, not exposed to the client

14. Scalability Notes

Although the initial deployment targets a modest scale, the design anticipates future growth in two ways: the FAQ embedding lookup can be moved to a dedicated vector-search index if the dataset grows very large, and the API layer is kept stateless so that additional server instances can be added behind a load balancer without design changes.

15. Design Review Checklist

Before the design was approved for development, it was reviewed against the following checklist to confirm readiness:

- Every functional requirement from the Requirement Analysis document maps to at least one module or endpoint
- Every database table has a clear purpose and clearly defined key relationships
- Every admin-only endpoint has a documented access-control rule
- Error scenarios are defined for each major endpoint, not only the success path

16. Mapping Requirements to Design Elements

Requirement	Design Element Addressing It
FR-1, FR-2, FR-3, FR-4	POST /api/ask endpoint, Query Handler, NLP Matching Engine (Sections 4, 6, 7)
FR-5, FR-6, FR-7	FAQ Management module and /api/faqs endpoints (Sections 4, 6)
FR-8	categories table and category_id relationship in faqs table (Section 5)
FR-9	query_logs table and Analytics/Logging module (Sections 4, 5)
FR-10	feedback table and Feedback Module (Sections 4, 5)
FR-11	Auth Module and AuthMiddleware (Sections 4, 10, 12)

17. Conclusion

The design described in this document provides a modular, secure, and scalable foundation for the AI FAQ Assistant API. It directly maps back to the functional and non-functional requirements defined earlier, and will guide the work carried out in the Project Planning.